

PROJECT DOCUMENT

Hospital Management System (HMS)

Project: Hospital Management System – Full Stack Implementation

Document Type: Technical Architecture & Implementation Record

Date: July 2026

1. Project Overview

The Hospital Management System (HMS) is a full-stack digital platform developed to streamline and automate core hospital operations. It consists of three integrated components: a Node.js/Express backend, an Angular web frontend, and a React Native mobile application. Together, these components provide a comprehensive, role-based system for managing patients, employees, appointments, medical records, and system access.

1.1 Purpose

The HMS platform serves as the central system for hospital operations. The backend functions as the API server, the web frontend provides the administrative and staff interface, and the mobile application delivers a patient-facing experience on smartphones. All three layers are tightly integrated through a shared API design and role-based authorization model.

1.2 System Components

- Backend (Node.js / Express): Provides RESTful APIs for all business operations including authentication, patient management, appointment handling, and medical records.
- Frontend (Angular 21): Web-based management interface for administrators, receptionists, and doctors to manage all HMS operations.
- Mobile Application (React Native / Expo): Patient-facing mobile app for appointment booking, medical history viewing, and profile management.

1.3 Business Alignment

This implementation directly supports the business objectives defined in the HMS Business Requirements Document (BRD), including:

- Role-based CRUD operations across all modules
- Approval workflows for employee self-signup and patient appointment requests
- Secure authentication using JWT tokens
- Role-based page and API access control
- Comprehensive management of Employee, Patient, Appointment, and Health Record modules

2. Backend Architecture

The backend is a Node.js and Express.js application that acts as the central API server for the HMS platform. It manages authentication, authorization, data access, and business logic for all system operations.

2.1 Technology Stack

Technology	Purpose
Node.js	Server-side JavaScript runtime
Express.js	Web framework for routing and middleware
MongoDB	NoSQL database for all data storage
Mongoose	ODM library for MongoDB schema management
JSON Web Tokens (JWT)	Authentication and authorization tokens
express-validator	Request input validation
Brevo API	Email notification and verification

Technology	Purpose
Helmet	HTTP security headers middleware
Morgan	HTTP request logging
CORS	Cross-Origin Resource Sharing control

2.2 Architecture Design

The backend follows a layered architecture pattern with clear separation of concerns across five distinct layers:

2.2.1 Entry Layer

The application is initialized in `src/app.js` and `src/server.js`. All global middleware including CORS, Helmet, JSON parsing, Morgan logging, and cookie parsing are registered at this level.

2.2.2 Routing Layer

Route modules in `src/routes` define the available API endpoints. Each file groups endpoints by business domain — authentication, profile, appointments, menu nodes, dashboard, employees, patients, email verification, roles, permissions, medical records, and departments.

2.2.3 Controller Layer

Controllers in `src/controller` contain all request handling logic. They receive HTTP requests, apply validation and business checks, interact with Mongoose models, and return JSON responses.

2.2.4 Model Layer

Mongoose models in `src/models` define the MongoDB schema and data structures. They include validation rules, indexes, and pre-save hooks for auto-generated IDs.

2.2.5 Middleware and Validation Layer

Middleware in `src/middleware` handles authentication, permission checking, validation, and centralized error handling. Validation rules are defined separately in `src/validation` and attached to routes.

2.3 Folder Structure

Folder	Purpose
<code>src/controller</code>	Request handlers for each functional area
<code>src/middleware</code>	Reusable middleware for auth, permissions, validation, and error handling
<code>src/models</code>	Mongoose schemas defining all database collections
<code>src/routes</code>	API endpoint definitions grouped by business domain
<code>src/utlis</code>	Shared helpers: error factory, async handler, mail utility, token generator
<code>src/validation</code>	express-validator rule sets attached to routes

2.4 API Design

The API design below has been updated to match the authoritative OpenAPI specification for the backend. Route prefixes are shown without the `"/api"` mount prefix, consistent with the rest of this document; the OpenAPI spec mounts these same routes under `"/api"` at the application level (see note at the end of this section).

The backend exposes REST APIs mounted on the following base paths:

- `/auth` — Authentication and token management
- `/profile` — Authenticated user's own profile
- `/appointment` — Appointment lifecycle management

- /menuNode — Role-based sidebar/navigation node metadata and permission checks
- /dashboard — Dashboard statistics and employee overview
- /employees — Employee account operations, including approval/rejection
- /patients — Patient profile operations, including mobile self-registration
- /email — Email verification
- /roles — Role operations for RBAC
- /permissions — Permission operations for action-based permission control
- /records — Medical record CRUD operations
- /departments — Department operations

2.4.1 Authentication Endpoints

Method	Route	Description
POST	/auth/signupByUser	Register a new user via admin or user signup flow
POST	/auth/login	Authenticate user credentials and return access token
POST	/auth/refresh	Refresh access token using refresh token
POST	/auth/logout	Logout user and clear session / refresh token
POST	/auth/setpassword	Set or change first-time password
POST	/auth/forgot-password	Request a password reset email
GET	/auth/reset-password	Verify password reset token and complete the reset flow

2.4.2 Profile Endpoints

Method	Route	Description
GET	/profile/me	Get the current authenticated user's profile

2.4.3 Appointment Endpoints

Method	Route	Description
POST	/appointment/create	Create a new appointment
GET	/appointment/stats	Get appointment statistics
GET	/appointment/doctors	Fetch doctors list for appointment booking
GET	/appointment/recent	Retrieve recent appointments
PUT	/appointment/:id	Update an appointment by ID
DELETE	/appointment/:id	Delete an appointment by ID
GET	/appointment/slots	Get available appointment slots
GET	/appointment/my-appointments	Get the authenticated user's own appointments
GET	/appointment/all	Get all appointments

2.4.4 Menu Node Endpoints

Method	Route	Description
POST	/menuNode/createMenuNode	Create a new menu node
PUT	/menuNode/updateMenuNode/:id	Update a menu node by ID
DELETE	/menuNode/deleteMenuNode/:id	Delete a menu node by ID
GET	/menuNode/getSidebarMenu	Get the authenticated user's sidebar menu
GET	/menuNode/getMenus	List all menu nodes
GET	/menuNode/check-permission/:path	Check whether the current user has permission for a UI path

2.4.5 Dashboard Endpoints

Method	Route	Description
GET	/dashboard/stats	Get dashboard statistics
GET	/dashboard/tenEmployees	Get employee overview for the dashboard

2.4.6 Employee Endpoints

Method	Route	Description
GET	/employees/all	Get all employees
POST	/employees/create	Create a new employee
PUT	/employees/:id	Update employee data by ID
DELETE	/employees/:id	Delete an employee by ID
PATCH	/employees/approve/:id	Approve a pending employee
PATCH	/employees/reject/:id	Reject a pending employee

2.4.7 Patient Endpoints

Method	Route	Description
GET	/patients/all	Get all patients
POST	/patients/create	Create a patient from the admin interface
PUT	/patients/:id	Update a patient by ID
DELETE	/patients/:id	Delete a patient by ID
POST	/patients/mobile-register	Register a new patient from the mobile app (public/mobile route)

2.4.8 Email Verification Endpoints

Method	Route	Description
GET	/email/verify-email	Verify a user's email via the emailed verification token link

2.4.9 Role Endpoints

Method	Route	Description
POST	/roles/create	Create a new role
GET	/roles/show	List all roles
GET	/roles/public	Get public roles
PUT	/roles/:id	Update a role by ID
DELETE	/roles/:id	Delete a role by ID

2.4.10 Permission Endpoints

Method	Route	Description
POST	/permissions/	Create a permission
GET	/permissions/	Get all permissions
POST	/permissions/assign	Assign a permission to a role
POST	/permissions/revoke	Revoke a permission from a role

2.4.11 Medical Record Endpoints

Method	Route	Description
POST	/records/createRecord	Create a medical record
PUT	/records/updateRecord/:id	Update an existing medical record by ID
DELETE	/records/deleteRecord/:id	Delete a medical record by ID
GET	/records/getAllRecords	Get all medical records

Method	Route	Description
GET	/records/getMyRecords	Get the authenticated user's medical records
GET	/records/getRecord/:id	Get a single medical record by ID
GET	/records/getPatientRecords	Get the patient's own medical records

2.4.12 Department Endpoints

Method	Route	Description
POST	/departments/	Create a new department
GET	/departments/	List all departments
PUT	/departments/:id	Update a department by ID
DELETE	/departments/:id	Delete a department by ID

2.5 Database Design

The backend uses MongoDB as its database, accessed through Mongoose. The following collections form the core data model:

2.5.1 Data Models

Model	Collection Purpose	Key Fields
User	Login credentials and account state for all users	email, passwordHash, role, status, employeeId, patientId, isVerified, firstLogin
Employee	Staff and doctor profile information	employeeCode, name, department, designation, specialization, availabilitySlots, isDeleted
Patient	Patient profile information	uhid, name, phone, email, gender, dob, bloodGroup, allergies, address, emergencyContact
Appointment	Doctor-patient appointment records	appointmentId, patientId, doctorEmployeeId, date, timeSlot, status
Medical Record	Diagnosis and treatment details	medicalRecordId, doctorId, appointmentId, patientId, complaint, diagnosis, medications, isDeleted
Role	Role definitions and permissions	role_id, role_name, role_permissions
Department	Department reference data	department_id, department_name
Specialization	Doctor specialization reference data	specialization_id, specialization_name
Menu Node	Frontend navigation definitions by role	order, name, path, role, icon
Counter	Auto-increment sequences for IDs	name, seq

2.5.2 Entity Relationships

- User → Employee: a user may be linked to an employee record via employeeId
- User → Patient: a user may be linked to a patient record via patientId
- Appointment → Patient: each appointment belongs to one patient
- Appointment → Doctor: each appointment is assigned to one doctor employee
- Medical Record → Appointment: each record is linked to a specific appointment
- Medical Record → Patient & Doctor: each record also references the patient and treating doctor
- Role → Permissions: a role has a list of permissions that determine API and page access

2.6 Middleware

Middleware	File	Purpose
Authentication	src/middleware/auth.middleware.js	Verifies JWT Bearer token from the Authorization header and attaches decoded user to req.user
Permission	src/middleware/permission.middleware.js	Checks whether the authenticated user's role has the required permission for the route
Validation	src/middleware/validate.middleware.js	Uses express-validator to inspect request data and returns a 422 error if validation fails
Error Handler	src/middleware/errorHandler.middleware.js	Catches thrown errors and returns a consistent JSON error response with status and message

2.7 Request Lifecycle

A typical API request moves through the backend in the following order:

- 1. Client sends an HTTP request to a defined route
- 2. Validation middleware checks request data format and required fields
- 3. Authentication middleware verifies the JWT token if the route is protected
- 4. Permission middleware checks the user's role has access to the route
- 5. Controller function executes the business logic
- 6. Controller reads or writes data through Mongoose models
- 7. Controller returns a structured JSON response
- 8. Error handler middleware catches and formats any thrown errors

3. Frontend Architecture

The frontend is an Angular 21 web application that serves as the management interface for hospital staff. It provides role-based access to all HMS operational modules including dashboard, employee management, patient management, appointments, approvals, and medical records.

3.1 Technology Stack

Technology	Purpose
Angular 21	Core frontend framework
TypeScript	Strongly typed component and service development
Angular Router	Client-side navigation and route protection
Angular HttpClient	HTTP communication with the backend API
Angular Reactive Forms	Form management and validation
Angular Material Autocomplete	Enhanced input components
RxJS	Reactive programming and HTTP stream handling
ngx-toastr	User-facing toast notification messages
Bootstrap Icons	Icon library for UI elements
LocalStorage	Client-side storage for JWT tokens, role, and permissions

3.2 Application Structure

Folder	Purpose
src/app/theme/login	Login page component
src/app/theme/signup	Employee registration page
src/app/theme/home	Main application shell with header, sidebar, and router outlet
src/app/theme/home/dashboard	Dashboard statistics page
src/app/theme/home/appointment	Appointment management page
src/app/theme/home/employee	Employee list and management page
src/app/theme/home/patient	Patient management page
src/app/theme/home/user-profile	Logged-in user profile page
src/app/theme/home/approval	Pending user approval page
src/app/theme/home/medical-record	Medical record management page
src/app/theme/home/modal	Modal and route-based detail/edit components
src/app/services	Angular services for API communication and state management
src/app/models	TypeScript interfaces and data models
src/app/validators	Custom Angular form validators
src/app/directive	Custom Angular directives including permission-based UI control
src/app/environment	Environment configuration and backend base URL

3.3 Pages and Components

3.3.1 Authentication Pages

Page	File	Purpose
Login	src/app/theme/login/login.ts	Authenticates users via email and password, stores JWT and role in localStorage, and redirects to the appropriate page
Sign Up	src/app/theme/signup/signup.ts	Allows new employee users to register. Loads roles, departments, and specializations from the backend

3.3.2 Core Application Pages

Page	File	Purpose
Home Shell	src/app/theme/home/home.ts	Main layout container with header, sidebar, and router outlet. Redirects unauthenticated users to login
Dashboard	src/app/theme/home/dashboard/dashboard.ts	Displays system statistics loaded from the dashboard API
Appointment	src/app/theme/home/appointment/appointment.ts	Appointment creation, update, deletion, and status management with doctor availability filtering
Employee	src/app/theme/home/employee/employee.ts	Employee list with pagination, department filtering, search, and delete/edit navigation
Patient	src/app/theme/home/patient/patient.ts	Patient registration, listing, search, editing, and deletion
User Profile	src/app/theme/home/user-profile/user-profile.ts	Displays the logged-in user's profile loaded from the backend
Approval	src/app/theme/home/approval/approval.ts	Admin review of pending user accounts with approve or reject actions
Medical Record	src/app/theme/home/medical-record/medical-record.ts	Medical record creation, update, draft and completed states, paginated list

3.3.3 Modal and Detail Components

Component	Purpose
modal/password-modal	First-login password setup flow
modal/edit-employee	Edit employee profile form
modal/edit-patient	Edit patient profile form
modal/view-medical-record	View a full medical record in detail
modal/access-denied	Displayed when a user lacks permission for a page
modal/not-found	404 not found page

3.4 Services and API Integration

All backend communication is handled through Angular services using HttpClient. The base API URL is configured in the environment file.

Service	Backend Endpoints Used	Responsibility
auth.service.ts	/auth	Login, signup, password setup, token refresh
user.service.ts	/profile, /patients	User profile, patient management, search, profile update
appointment.service.ts	/appointment	Create, list, edit, delete, and query appointments
admin.service.ts	/dashboard, /employees	Dashboard statistics, employee data, approval actions, profile updates
medical-record.service.ts	/records	Medical record creation, update, retrieval, and deletion

3.4.1 Authentication Interceptor

The auth interceptor (src/app/services/interceptor/auth.interceptor.ts) automatically attaches the JWT access token as an Authorization header to all outgoing API requests. It also handles token refresh when the access token expires.

3.4.2 Route Guard

The route guard (src/app/services/guard/route.guard.ts) checks the logged-in user's permissions before allowing navigation to protected pages. If access is denied, the user is redirected to the access-denied page.

3.5 User Flows

3.5.1 Login and Role-Based Navigation

- User opens the login page and submits email and password
- Angular login component sends credentials to the backend through the auth service
- On success, the app stores the JWT token, email, role, and permissions in localStorage
- The home shell loads the sidebar dynamically based on menu node data from the backend
- The route guard checks permissions before allowing access to each protected page

3.5.2 Admin Flow

- Admin logs in and sees dashboard statistics
- Admin can manage employees, patients, and review pending approval requests
- Admin can approve or reject user accounts and update employee profiles

3.5.3 Staff and Doctor Flow

- Staff logs in and accesses operational pages based on role permissions
- Receptionists can create and manage appointments and patients
- Doctors can manage medical records and record diagnosis details for completed appointments

4. Mobile Application Architecture

The frontend mobile client is an Expo-based React Native application built with TypeScript, and is the patient-facing client for the HMS platform. It allows patients to log in, browse doctors, book and manage appointments, view medical records, and update their profile from a smartphone. The project uses React Navigation for authentication and app navigation, with a native stack navigator for the login/register flow and a bottom tab navigator for the main application screens. The UI is composed of reusable React Native components, custom cards, forms, and screens organized under the src structure. Form validation is implemented using React Hook Form and Yup. API communication is managed through an Axios-based client with interceptors for authentication and error handling. Authentication tokens and patient profile data are stored securely using Expo Secure Store. The application is configured for Android, iOS, and web through Expo, with support for custom fonts, icons, and toast notifications.

4.1 Technology Stack

Technology	Purpose
React Native	Cross-platform mobile framework
Expo	Development toolchain, build platform, and Android/iOS/web configuration (app.json)
TypeScript	Strongly typed component and service development
React Navigation (Native Stack)	Authentication flow navigation (login/register)
React Navigation (Bottom Tab)	Main app tab navigation
React Hook Form	Form state management and submission handling
Yup	Schema-based form validation, integrated with React Hook Form
Axios	Centralized HTTP client for backend API communication
Expo Secure Store	Secure on-device storage for JWT tokens and patient profile data
Expo Vector Icons	Icon library for mobile UI
React Native Toast Message	User-facing feedback messages
DateTimePicker	Date and time selection in forms
Expo Font	Custom font loading

4.2 Application Structure

The project follows a modular, screen-driven folder structure rather than a component-first or feature-first-only layout. Business logic is separated into dedicated service modules, navigation and API interceptors live in their own folders, and shared types are centralized under src/types. This separation keeps UI, API integration, routing, and authentication concerns organized.

Folder / File	Purpose
index.ts	Expo application entry point; registers the root component and boots the app runtime
App.tsx	Root app component: loads fonts, mounts the navigation container, and renders the global toast provider
src/navigation/AppNavigator.tsx	Root native stack navigator for the authentication flow (login, signup)
src/navigation/MainTabNavigator.tsx	Bottom tab navigator for the main app shell (Home, Profile, Appointment, Record), with nested stack navigation for appointment screens
src/screens	All application screens (login, signup, home, appointment, profile, medical records)
src/components	Reusable UI blocks: input cards, profile cards, doctor cards, form headers, auth buttons
src/services	Service modules for backend communication, including the centralized Axios client and authentication service
src/hooks	Custom hooks for shared form and business logic

Folder / File	Purpose
src/validations	Yup validation schemas used by React Hook Form
src/utils	Shared helper functions
src/types	Shared TypeScript interfaces for users, appointments, medical records, auth, and navigation

4.3 Entry and Startup Flow

The app entry is handled through Expo in `index.ts`, rather than a traditional bare React Native bootstrap. `index.ts` registers the root component with Expo's runtime, which then mounts `App.tsx` — the root app component responsible for loading custom fonts, mounting the React Navigation container, and rendering the global toast provider before handing control to the navigators.

4.4 Screens

Screen	File	Purpose
Login	src/screens/login.tsx	Authenticates a patient using email and password and redirects to the main app shell
Sign Up	src/screens/signup.tsx	Registers a new patient account and captures personal and contact details
Home	src/screens/home.tsx	Doctor browsing and search with specialization filters
Appointment	src/screens/appointment.tsx	Book a new appointment by selecting a doctor, date, and available time slot
View Appointment	src/screens/view-appointment.tsx	Lists all appointments for the logged-in patient
Edit Appointment	src/screens/edit-appointment.tsx	Modify or cancel an existing appointment
Profile	src/screens/profile.tsx	View patient personal information and account actions
Edit Profile	src/screens/edit-profile.tsx	Update profile data including name, gender, address, and emergency contact
View Medical Record	src/screens/view-medical-record.tsx	Paginated medical record history with expandable details for complaints, symptoms, diagnosis, and medications

4.5 Navigation Structure

The application uses React Navigation with a nested structure: a native stack navigator for the authentication flow, a bottom tab navigator for the main app shell, and a nested stack navigator for appointment screens.

4.5.1 Stack Navigation (Root)

The root stack navigator, defined in `App.tsx / src/navigation/AppNavigator.tsx`, includes: login, signup, tabs (main app shell), `viewAppointment`, `editAppointment`, and `editProfile`.

4.5.2 Bottom Tab Navigation

The tab navigator, defined in `src/navigation/MainTabNavigator.tsx`, provides four main sections accessible from the app bottom bar, with appointment screens further nested in their own stack:

- Home — Doctor browsing and appointment initiation
- Profile — Patient profile viewing and editing
- Appointment — Appointment listing and management (nested stack: `appointment`, `view-appointment`, `edit-appointment`)
- Record — Medical record history

4.6 API Layer

Backend communication is handled through a centralized Axios client in `src/services/apiClient.ts`, rather than ad hoc requests scattered across screens. The client is configured with interceptors that attach the JWT bearer token to protected requests and handle authentication and error responses consistently across the app. The endpoints below reflect the base paths defined in Section 2.4.

4.6.1 Authentication Endpoints

Endpoint	Purpose
<code>auth/login</code>	Authenticate a patient with email and password
<code>patients/mobile-register</code>	Register a new patient account from the mobile app

4.6.2 Profile Endpoints

Endpoint	Purpose
<code>profile/me</code>	Fetch the patient's own profile details
<code>patients/:id</code>	Submit updated patient profile data (PUT)

4.6.3 Appointment Endpoints

Endpoint	Purpose
<code>appointment/create</code>	Create a new appointment for the patient
<code>appointment/my-appointments</code>	Retrieve all appointments for the logged-in patient
<code>appointment/:id</code>	Modify appointment details, or cancel/change status (PUT/DELETE)
<code>appointment/doctors</code>	Retrieve the list of available doctors
<code>appointment/slots</code>	Fetch available appointment time slots for a doctor on a selected date

4.6.4 Medical Record Endpoints

Endpoint	Purpose
<code>records/getPatientRecords</code>	Retrieve paginated medical records for the patient

4.7 Authentication Flow

Login and registration are handled through `src/services/authService.ts`, which wraps the `auth/login` and `patients/mobile-register` endpoints. On successful authentication, the JWT access token and the patient's profile data are persisted using Expo Secure Store rather than plain `AsyncStorage` or a custom storage layer, so sensitive session information stays encrypted on the device rather than in plain local storage.

4.8 Storage Strategy

Authentication state is persisted locally using Expo Secure Store. The app stores the JWT access token and patient profile data securely on the device to support session continuity across app launches and secure logout behavior. There is no local database layer such as SQLite or Realm, and `AsyncStorage` is not used as the primary persistence mechanism for session data.

Data	Storage Mechanism	Reason
JWT Access Token	Expo Secure Store	Encrypted, on-device storage for sensitive authentication tokens
Patient Profile Data	Expo Secure Store	Encrypted, on-device storage to support session continuity across app launches

4.9 Form Validation

Form state is managed with React Hook Form, with validation schemas defined in Yup and applied before any request is sent to the backend:

Form	Validated Fields
Login	Email format, password minimum length
Sign Up	Name, email, password, confirm password match, phone, gender, date of birth, address, emergency contact
Appointment Booking	Doctor selection, date selection, time slot selection
Edit Profile	Name, gender, date of birth, address, emergency contact format

4.10 Shared Logic

The custom hook `src/hooks/useAppointmentForm.ts` centralizes shared appointment logic across the booking and editing screens:

- Doctor list loading from the backend
- Patient ID retrieval from the authenticated session
- Date handling and formatting
- Available time slot fetching based on selected doctor and date
- Appointment form validation before submission (React Hook Form + Yup)
- Navigation back to the appointment list after successful booking

4.11 Platform Support

The application is configured for Android, iOS, and web through Expo's `app.json` configuration, rather than being purely mobile-only. Shared platform-level UI concerns include custom Expo fonts, vector icons, safe-area handling, and toast notifications, all composed through reusable components rather than duplicated per screen.

5. System Integration

All three components — the backend, the web frontend, and the mobile application — are integrated through a shared RESTful API. The backend is the single source of truth for all data and business logic.

5.1 Integration Overview

Component	Communicates With	Protocol	Auth Mechanism
Angular Frontend	Node.js Backend	HTTPS REST API	JWT Bearer Token via HTTP Interceptor
React Native Mobile App	Node.js Backend	HTTPS REST API	JWT Bearer Token via Axios Interceptor
Backend	MongoDB	Mongoose ODM	Internal connection string

5.2 Authentication Flow Across Components

- A user or patient logs in through either the web frontend or mobile app
- Credentials are sent to the /auth/login endpoint (patients register via /patients/mobile-register)
- The backend validates credentials, generates a JWT access token and refresh token, and returns them
- The web frontend stores the token in localStorage; the mobile app stores it in Expo Secure Store
- All subsequent protected requests attach the token as an Authorization: Bearer header
- The backend auth middleware validates the token on each request
- The permission middleware checks whether the user's role permits the requested action

5.3 Role-Based Access Across Layers

Role	Web Frontend Access	Mobile App Access
Super Admin	Full access to all modules including employee creation, deletion, approvals, and all settings	Not applicable — admin roles use web interface
Admin	Employee management, patient management, appointment management, dashboard, approvals, medical records	Not applicable
Receptionist	Patient registration and management, appointment creation and updates, medical record access	Not applicable
Doctor	Appointment viewing, medical record creation and update for assigned patients	Not applicable
Patient	Not applicable — patients use mobile app	Login, doctor search, appointment booking and management, medical record viewing, profile management

5.4 Data Consistency

Because all clients (web and mobile) communicate with the same backend and MongoDB instance, data is consistent across all access channels in real time. A patient appointment created on the mobile app is immediately visible to a receptionist on the web interface.

5A. Deployment & DevOps Architecture

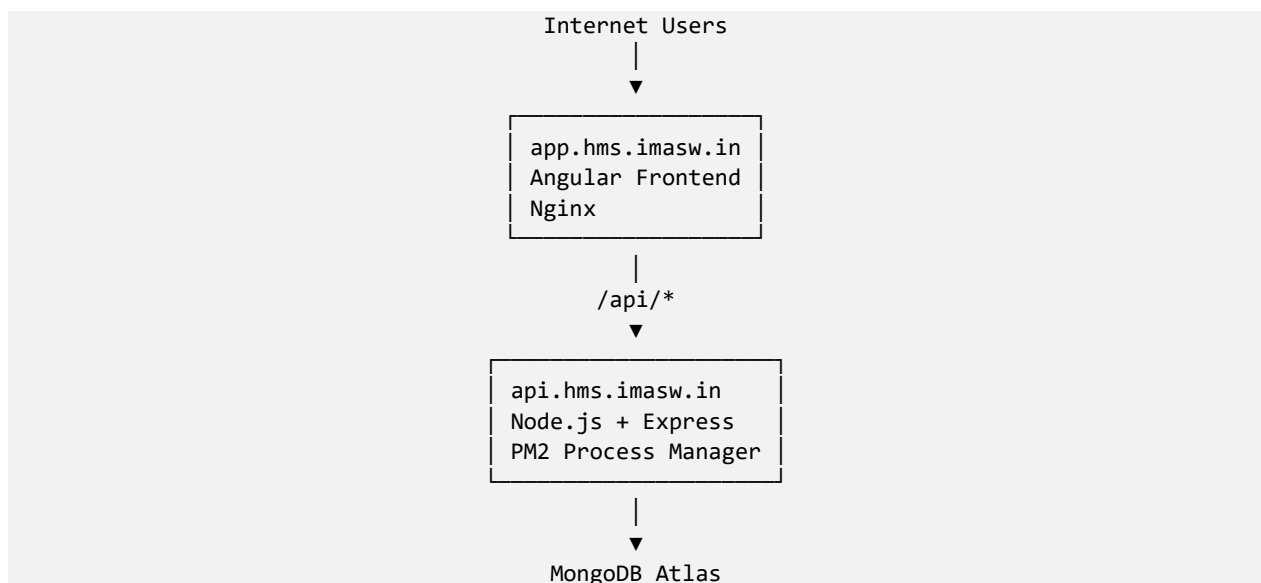
5A.1 Production Infrastructure Overview

The HMS platform is deployed on Amazon Web Services (AWS) using separate EC2 instances for the frontend and backend applications.

Production Domains

Component	Domain
Angular Frontend	https://app.hms.imasw.in
Node.js Backend API	https://api.hms.imasw.in
Swagger Documentation	https://api.hms.imasw.in/api-docs

Hosting Architecture



5A.2 AWS EC2 Deployment

Frontend EC2

Item	Value
Purpose	Angular Application
Web Server	Nginx
Runtime	Node.js
Deployment Path	/home/ubuntu/hms-frontend-pod3
Published Files	/var/www/html

Backend EC2

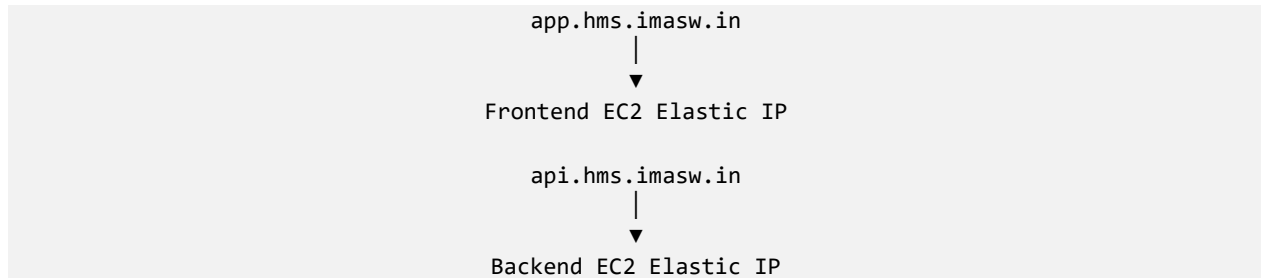
Item	Value
Purpose	Express API
Runtime	Node.js
Process Manager	PM2
Deployment Path	/home/ubuntu/HMS_PROJECT/hms-backend-pod-3

Item	Value
API Port	5000

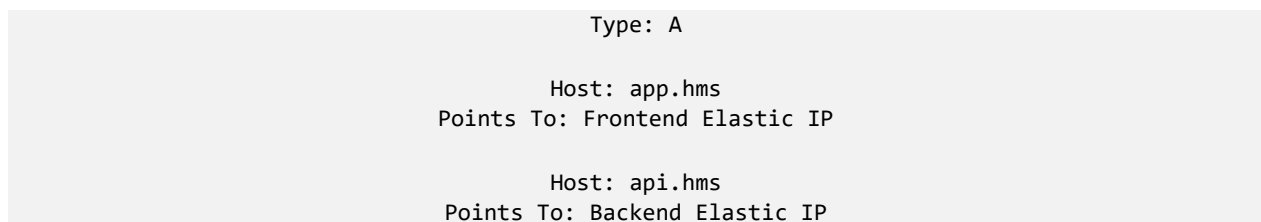
5A.3 Domain Configuration

The domain imasw.in is registered through GoDaddy.

DNS records are configured as follows:



DNS Record Example:



Elastic IPs are used to prevent DNS changes when EC2 instances are restarted.

5A.4 SSL Certificate Configuration

SSL certificates are provisioned using Let's Encrypt and Certbot.

Install Certbot

```

sudo apt update
sudo apt install certbot python3-certbot-nginx -y
  
```

Generate Certificate

Frontend:

```

sudo certbot --nginx -d app.hms.imasw.in
  
```

Backend:

```

sudo certbot --nginx -d api.hms.imasw.in
  
```

Verify:

```

curl -I https://app.hms.imasw.in
curl -I https://api.hms.imasw.in
  
```

5A.5 Frontend Nginx Configuration

The frontend Angular application is deployed as static files and served via Nginx.

Location:

```

/etc/nginx/sites-available/default
  
```

Configuration:

```

server {

    server_name app.hms.imasw.in;

    root /var/www/html;
    index index.html;

    location /api/ {
        proxy_pass http://api.hms.imasw.in:5000;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location / {
        try_files $uri $uri/ /index.html;
    }
}

```

5A.6 Angular Production Build Deployment

Application build command:

```
npm run build
```

Output:

```
dist/HMS-FRONTEND/browser
```

Deployment Process:

```
sudo rm -rf /var/www/html/*
```

```
sudo cp -r dist/HMS-FRONTEND/browser/* /var/www/html/
```

Angular SSR generates:

```
index.csr.html
```

which must be copied to:

```
index.html
```

Deployment command:

```
sudo cp /var/www/html/index.csr.html /var/www/html/index.html
```

Reload Nginx:

```
sudo nginx -t
```

```
sudo systemctl reload nginx
```

5A.7 Backend Deployment

Install Dependencies

```
npm ci
```

Restart Application

```
sudo pm2 restart healthcare-api
```

```
sudo pm2 save
```

Verify Status

```
sudo pm2 list
```

Expected:

```
healthcare-api    online
```

5A.8 GitHub Actions CI/CD Pipeline

The HMS project uses GitHub Actions for automated deployment.

Deployment branch:

```
feature/sprint_5/aswin
```

GitHub Repository Secrets

Frontend Repository:

```
FRONTEND_HOST
```

```
SSH_USER
```

```
SSH_PRIVATE_KEY
```

Example:

```
FRONTEND_HOST=app.hms.imasw.in
```

```
SSH_USER=ubuntu
```

Backend Repository:

```
BACKEND_HOST
```

```
SSH_USER
```

```
SSH_PRIVATE_KEY
```

Example:

```
BACKEND_HOST=api.hms.imasw.in
```

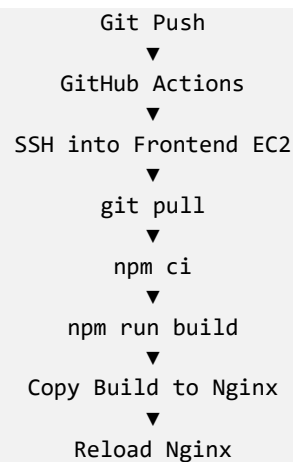
```
SSH_USER=ubuntu
```

5A.9 Frontend CI/CD Workflow

Location:

```
.github/workflows/deploy.yml
```

Deployment Flow:



5A.10 Backend CI/CD Workflow

Deployment Flow:



5A.11 GitHub Authentication on EC2

A dedicated SSH key is configured for GitHub Actions deployment.

Generate Key:

```
ssh-keygen -t ed25519 -C "github-actions"
```

Public key:

```
id_ed25519.pub
```

added to:

```
/home/ubuntu/.ssh/authorized_keys
```

on both EC2 instances.

5A.12 Deployment Validation Commands

Frontend

```
curl -I https://app.hms.imasw.in
```

Expected:

```
HTTP/1.1 200 OK
```

Backend

```
curl https://api.hms.imasw.in
```

Expected:

```
{
  "message": "API running"
}
```

PM2

```
sudo pm2 list
```

Expected:

```
healthcare-api  online
```

5A.13 Production Deployment Steps

Frontend

```
name: Deploy Frontend

on:
  push:
    branches:
      - feature/sprint_5/aswin

concurrency:
  group: frontend-deploy
  cancel-in-progress: true

jobs:
  deploy:
    runs-on: ubuntu-latest

    steps:
      - name: Deploy Frontend to EC2
        uses: appleboy/ssh-action@v1.2.0
        with:
          host: ${ secrets.FRONTEND_HOST }
          username: ${ secrets.SSH_USER }
          key: ${ secrets.SSH_PRIVATE_KEY }
          script: |
            set -e

            mkdir -p ~/.ssh
            ssh-keyscan github.com >> ~/.ssh/known_hosts 2>/dev/null

            cd /home/ubuntu/hms-frontend-pod3

            git fetch origin
            git checkout feature/sprint_5/aswin
            git pull origin feature/sprint_5/aswin

            npm ci
            npm run build

            test -d dist/HMS-FRONTEND/browser

            sudo rm -rf /var/www/html/*

            sudo cp -r dist/HMS-FRONTEND/browser/* /var/www/html/

            if [ -f /var/www/html/index.csr.html ]; then
              sudo cp /var/www/html/index.csr.html /var/www/html/index.html
            fi

            sudo nginx -t
```

```
sudo systemctl reload nginx
```

```
echo "Frontend deployment completed successfully"
```

Backend

```
name: Deploy Backend
```

```
on:
```

```
  push:
```

```
    branches:
```

```
      - feature/sprint_5/aswin
```

```
concurrency:
```

```
  group: backend-deploy
```

```
  cancel-in-progress: true
```

```
jobs:
```

```
  deploy:
```

```
    runs-on: ubuntu-latest
```

```
  steps:
```

```
    - name: Deploy Backend to EC2
```

```
      uses: appleboy/ssh-action@v1.2.0
```

```
      with:
```

```
        host: ${ secrets.BACKEND_HOST }
```

```
        username: ${ secrets.SSH_USER }
```

```
        key: ${ secrets.SSH_PRIVATE_KEY }
```

```
        script: |
```

```
          set -e
```

```
          mkdir -p ~/.ssh
```

```
          ssh-keyscan github.com >> ~/.ssh/known_hosts 2>/dev/null
```

```
          cd /home/ubuntu/HMS_PROJECT/hms-backend-pod-3
```

```
          git fetch origin
```

```
          git checkout feature/sprint_5/aswin
```

```
          git pull origin feature/sprint_5/aswin
```

```
          npm ci
```

```
          sudo pm2 restart healthcare-api
```

```
          sudo pm2 save
```

```
          echo "Backend deployment completed successfully"
```

6. Functional Modules Summary

The following table summarizes the key functional modules and their implementation across all three platform layers:

Module	Backend Base Route	Web Frontend Page	Mobile Screen
Authentication	/auth	login.ts, signup.ts	login.tsx, signup.tsx
Profile	/profile	user-profile.ts	profile.tsx, edit-profile.tsx
Employee Management	/employees	employee.ts, approval.ts	Not applicable
Patient Management	/patients	patient.ts	profile.tsx, edit-profile.tsx
Appointment Management	/appointment	appointment.ts	appointment.tsx, view-appointment.tsx, edit-appointment.tsx
Medical Records	/records	medical-record.ts	view-medical-record.tsx
Dashboard & Statistics	/dashboard	dashboard.ts	Not applicable
Menu Node / Navigation	/menuNode	Dynamic sidebar	Not applicable
Roles	/roles	Role management (admin settings)	Not applicable
Permissions	/permissions	Permission management (admin settings)	Not applicable
Departments	/departments	Department management (admin settings)	Not applicable
Email Verification	/email	N/A — email link flow	N/A — email link flow

7. Error Handling and Validation Strategy

7.1 Backend Validation

Input validation is implemented using express-validator. Validation rules are attached to routes before the controller executes. Common validations include:

- Email format and uniqueness
- Required field presence
- Allowed role, department, and specialization values
- Date format and password strength requirements

7.2 Backend Error Handling

- Custom error objects are created through the centralized error factory in src/utils/errors.utils.js
- Controllers throw these custom errors instead of handling each case manually
- The asyncHandler wrapper ensures all unhandled promise rejections are forwarded to error middleware
- The error handler middleware formats all errors into a consistent JSON response with status code and message

7.3 Frontend Error Handling

- Catching HTTP failures in Angular services using RxJS error operators
- Displaying user-friendly toast notifications using ngx-toastr
- Redirecting users to the access-denied page for permission failures
- Redirecting users to the 404 not-found page for invalid routes

7.4 Mobile Application Error Handling

- The Axios interceptor displaying a user-friendly alert for all API error responses
- Local form validation providing inline feedback before any request is sent
- Toast messages confirming successful operations or communicating failures

7.5 Common Error Scenarios

Error Scenario	Backend Response	Client Handling
Invalid login credentials	401 Unauthorized	Toast notification: invalid email or password
Missing or expired JWT token	401 Unauthorized	Interceptor redirects user to login page
Insufficient permissions	403 Forbidden	Redirect to access-denied page
Invalid request body	422 Unprocessable Entity	Field-level validation messages displayed
Duplicate email or registration number	409 Conflict	Toast notification with error detail
Resource not found (patient, doctor, appointment)	404 Not Found	Toast notification or 404 page
Server or database error	500 Internal Server Error	Generic error toast message

8. Conclusion

The Hospital Management System has been implemented as a cohesive three-tier platform. The Node.js/Express backend provides a robust, layered API that enforces role-based authorization and maintains data integrity through structured Mongoose schemas. The Angular frontend delivers a responsive and secure management interface for all hospital staff roles. The React Native mobile application extends the platform to patients, enabling them to manage appointments and access their medical history from any mobile device.

The architecture is modular, maintainable, and aligned with the requirements defined in the HMS Business Requirements Document. Role-based access control is enforced at every layer — from route guards in the frontend to permission middleware on the backend — ensuring that all operations are performed only by authorized users. The shared API design between the web and mobile clients ensures data consistency across all access channels.